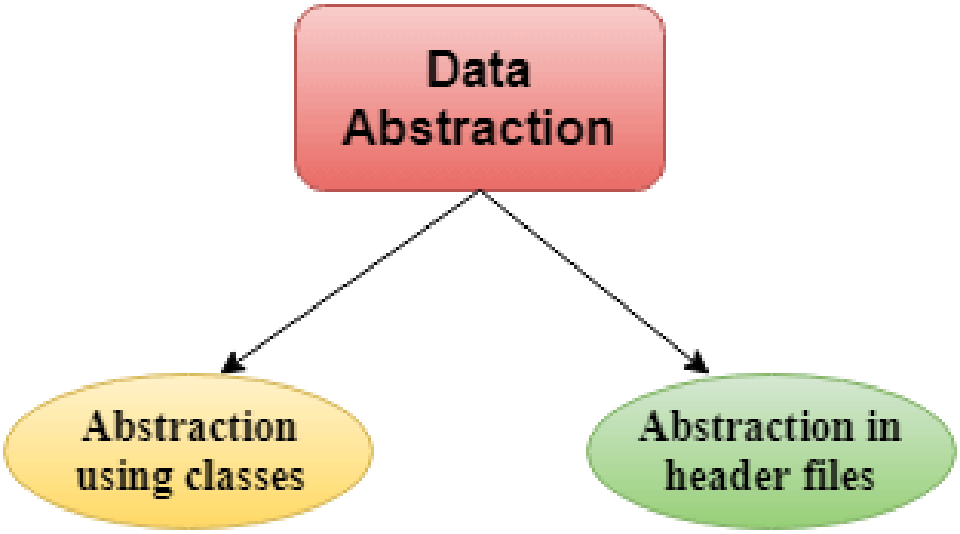
Data Abstraction in C++

Data Abstraction is a process of providing only the essential details to the outside world and hiding the internal details, i.e., representing only the essential details in the program. Data Abstraction is a programming technique that depends on the separation of the interface and implementation details of the program.

Let's take a real-life example of AC, which can be turned ON or OFF, change the temperature, change the mode, and other external components such as fan, swing. But we don't know the internal details of the AC, i.e., how it works internally. Thus, we can say that AC separates the implementation details from the external interface.

C++ provides a great level of abstraction. For example, pow () function is used to calculate the power of a number without knowing the algorithm the function follows.

Data Abstraction can be achieved in two ways:



Abstraction using classes: An abstraction can be achieved using classes. A class is used to group all the data members and member functions into aJ single unit by using the access specifiers. A class has the responsibility to determine which data member is to be visible outside and which is not.

Abstraction in header files: Another type of abstraction is header file. For example, pow() function available is used to calculate the power of a number without actually knowing which algorithm function to calculate the power. Thus, we can say that header files hide all the implementation details from the user.

Access Specifiers Implement Abstraction:

Public specifier: When the members are declared as public, members can be accessed anywhere from the program.

Private specifier: When the members are declared as private, members can only be accessed only by the member functions of the class.

Let's see a simple example of abstraction in header files:

```
#include <iostream>
#include<math.h>
using namespace std;
int main()
{
 int n = 4;
   int power = 3;
   int result = pow(n,power);        // pow(n,power) is the  power function
   std::cout << "Cube of n is : " <<result<< std::endl;
   return 0;
}
```

In the above example, pow() function is used to calculate 4 raised to the power 3. The pow() function is present in the math.h header file in which all the implementation details of the pow() function are hidden.

Output:

```
Cube of n is : 64
```

Let's see a simple example of data abstraction using classes:

```
#include <iostream>
using namespace std;
 class Sum
{
private: int x, y, z; // private variables
public:
void add()
{
cout<<"Enter two numbers: ";
cin>>x>>y;
z= x+y;
cout<<"Sum of two number is: "<<z<<endl;
}
};
int main()
{
Sum sm;
sm.add();
return 0;
}
```

In the above example, abstraction is achieved using classes. A class 'Sum' contains the private members x, y and z are only accessible by the member functions of the class.

Output:

```
Enter two numbers:
3
6
Sum of two number is: 9
```

Advantages Of Abstraction:

- Implementation details of the class are protected from inadvertent user level errors.
- A programmer does not need to write low-level code.
- Data Abstraction avoids code duplication, i.e., programmer does not have to undergo the same tasks every time to perform a similar operation.
- The main aim of the data abstraction is to reuse the code and the proper partitioning of the code across the classes.
- Internal implementation can be changed without affecting the user level code.

C++ Exception Handling

Exception Handling in C++ is a process to handle runtime errors. We perform exception handling so the normal flow of the application can be maintained even after runtime errors. All exceptions are derived from std::exception class. It is a runtime error which can be handled. If we don't handle the exception, it prints an exception message and terminates the program.

Advantage

It maintains the normal flow of the application. In such case, the rest of the code is executed even after exception.

In C++, we use 3 keywords to perform exception handling:

- try
- catch, and
- throw

The basic syntax for exception handling in C++ is given below:

```
try {

    // code that may raise an exception
    throw argument;
}

catch (exception) {
    // code to handle exception
}
```

Here, we have placed the code that might generate an exception inside the try block. Every try block is followed by the catch block.

When an exception occurs, the throw statement throws an exception, which is caught by the catch block.

The catch block cannot be used without the try block.

Let’s see an example of exception handling using try….catch block:

```
#include <iostream>
using namespace std;
int main() {
    double numerator, denominator, divide;
    cout << "Enter numerator: ";
    cin >> numerator;
    cout << "Enter denominator: ";
    cin >> denominator;
    try {
        // throw an exception if denominator is 0
        if (denominator == 0)
            throw 0;

        // not executed if denominator is 0
        divide = numerator / denominator;
        cout << numerator << " / " << denominator << " = " << divide << endl;
    }

    catch (int num_exception) {
        cout << "Error: Cannot divide by " << num_exception << endl;
    }
    return 0;
}
```

C++ Bitwise Operators

In C++, bitwise operators perform operations on integer data at the individual bit-level.

Here is a list of 6 bitwise operators included in C++.

Operator	Description
&	Bitwise AND Operator
|	Bitwise OR Operator
^	Bitwise XOR Operator
~	Bitwise Complement Operator
<<	Bitwise Shift Left Operator
>>	Bitwise Shift Right Operator

1. C++ Bitwise AND Operator

The bitwise AND & operator returns 1 if and only if both the operands are 1. Otherwise, it returns to 0.

a	b	a & b
0	0	0
0	1	0
1	0	0
1	1	1

Let's take a look at the bitwise AND operation of two integers 12 and 25:

```
12 = 00001100 (In Binary)

25 = 00011001 (In Binary)

//Bitwise AND Operation of 12 and 25

     00001100
&    00011001
     ________
     00001000  = 8 (In decimal)
```

```cpp
#include <iostream>
using namespace std;

int main() {
    // declare variables
    int a = 12, b = 25;

    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    cout << "a & b = " << (a & b) << endl;

    return 0;
}
```

2. C++ Bitwise OR Operator

The bitwise OR | operator returns 1 if at least one of the operands is 1. Otherwise, it returns 0.

a	b	a | b
0	0	0
0	1	1
1	0	1
1	1	1

```
12 = 00001100 (In Binary)
25 = 00011001 (In Binary)

Bitwise OR Operation of 12 and 25
    00001100
|   00011001
    __________
    00011101  = 29 (In decimal)
```

```cpp
#include <iostream>

int main() {
    int a = 12, b = 25;

    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    cout << "a | b = " << (a | b) << endl;

    return 0;
}
```

3. C++ Bitwise XOR Operator

The bitwise XOR (^) operator returns 1 if and only if one of the operands is 1. However, if both the operands are 0, or if both are 1, then the result is 0.

a	b	a ^ b
0	0	0
0	1	1
1	0	1
1	1	0

```
12 = 00001100 (In Binary)
25 = 00011001 (In Binary)

Bitwise XOR Operation of 12 and 25
    00001100
^   00011001
    __________
    00010101  = 21 (In decimal)
```

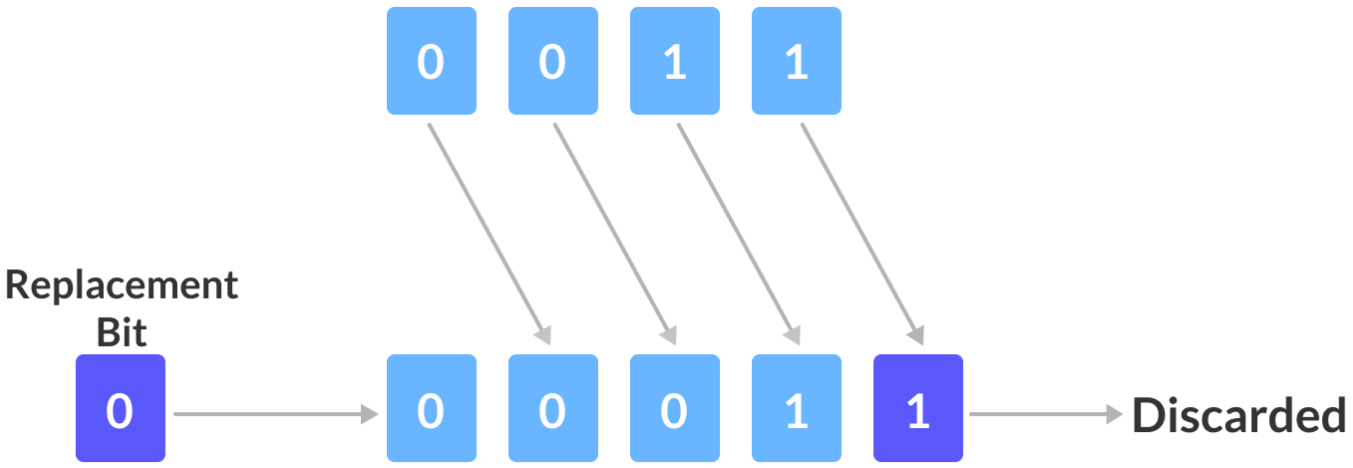
```cpp
#include <iostream>

int main() {
    int a = 12, b = 25;

    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    cout << "a ^ b = " << (a ^ b) << endl;

    return 0;
}
```

C++ Shift Operators

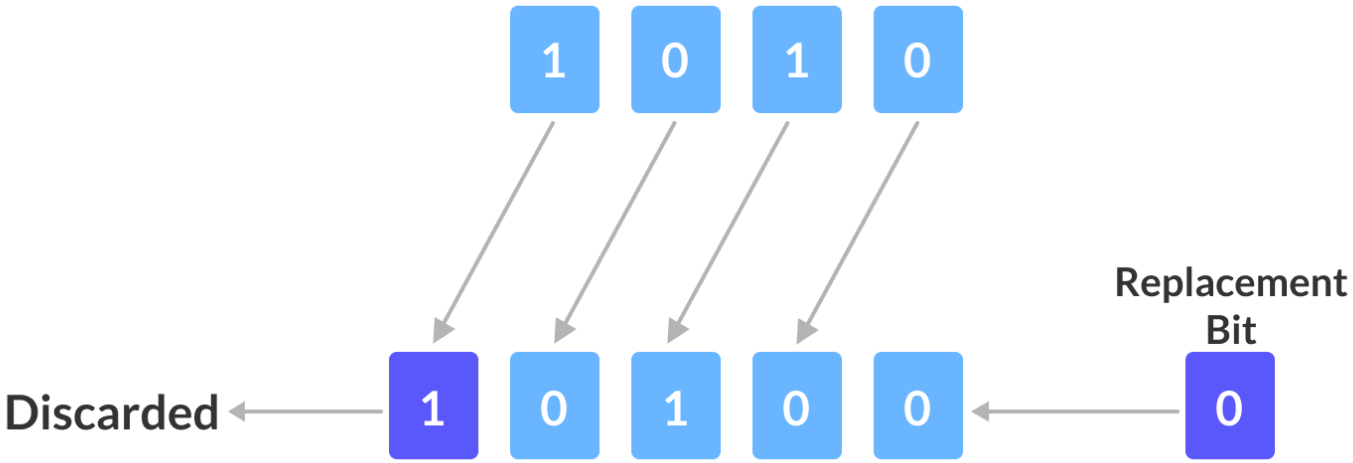
There are two shift operators in C++ programming:

- Right shift operator (>>)
- Left shift operator (<<)

Right Shift:



Left Shift:



Let’s see an example of right shift and left shift operator:

```
#include <iostream>

int main() {

    // declaring two integer variables
    int num = 212, i;

    // Shift Right Operation
    cout << "Shift Right:" << endl;

    // Using for loop for shifting num right from 0 bit to 3 bits
    for (i = 0; i < 4; i++) {
        cout << "212 >> " << i << " = " << (212 >> i) << endl;
    }

    // Shift Left Operation
    cout << "\nShift Left:" << endl;

    // Using for loop for shifting num left from 0 bit to 3 bits
    for (i = 0; i < 4; i++) {
        cout << "212 << " << i << " = " << (212 << i) << endl;
    }

    return 0;
}
```